00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

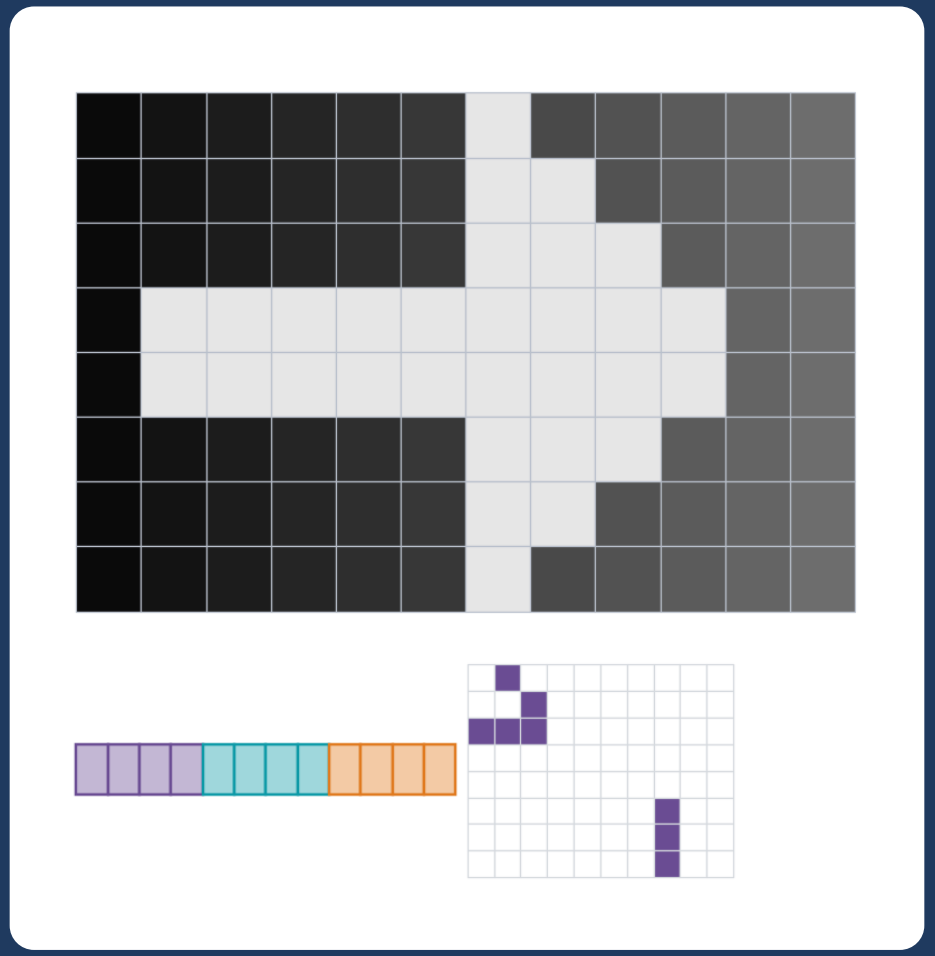
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 11

Diziler II: Çok Boyutlu Diziler ve Matris Düşüncesi

Satır öncelikli yerleşim · fonksiyona aktarım · matris işlemleri · ızgara ve komşuluk · görüntü filtreleri, Hayat Oyunu ve önbellek deneyi

MODÜL V · VERİ TOPLULUKLARI VE BELLEK



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** **Diziler II: Matrisler**	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H10 · İNDİS–ADRES

İlk satırda ne yazdırılır? (sizeof(int) = 4)

quiz/H11_Q01_indis_adres.c · satır 8–14

```c
int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};
    printf("%td\n", (char *)&a[3] - (char *)&a[0]);
    printf("%d\n", a[1] + a[4]);
    return 0;
}
```

A 3

B 12

C 24

D 40

Not: `(char *)` dönüşümü ve iki adresin farkı (`%td`) burada yalnızca bayt uzaklığını yazdırmanın yoludur; ayrıntısı H12'de (işaretçiler).

GERİ ÇAĞIRMA · H10

Geçen haftadan · S2 (diziyi fonksiyona aktarma) ve S3 (frekans dizisi)

S2. `main` 6 bulur. `dizi_max`'in içinde `sizeof a / sizeof a[0]` yazılsaydı (64 bit Linux) kaç olurdu?

quiz/H11_Q02_sizeof_tuzagi.c · satır 10

```
10  int dizi_max(const int a[], int n)
```

quiz/H11_Q02_sizeof_tuzagi.c · satır 23–26

```
23      int notlar[6] = {72, 95, 61, 88, 47, 90};
24      int n = (int)(sizeof notlar / sizeof notlar[0]);
25      printf("%d\n", n);
26      printf("%d\n", dizi_max(notlar, n));
```

- **A** 6
- **B** 1
- **C** 2
- **D** Derleme hatası

S3. Program ne yazdırır?

quiz/H11_Q03_frekans.c · satır 10–16

```
10      int atis[8] = {3, 6, 3, 1, 6, 3, 2, 5};
11      int frekans[7] = {0};        /* 1..6 */
12      for (int i = 0; i < 8; i++) {
13          frekans[atis[i]]++;      /* değer = indis */
14      }
15      printf("%d %d %d\n",
16             frekans[3], frekans[6], frekans[4]);
```

- **A** 3 2 0
- **B** 2 3 0
- **C** 3 2 1
- **D** 1 3 2

Yanıtlar: gerçek çıktılar

S1 · B 3 eleman × 4 bayt = **12 bayt**; uzaklık `(char *)` ile bayt cinsinden

GERÇEK ÇIKTI

```
$ ./q01
12
70
```

S2 · C Parametre bir **adrestir** (8 bayt): 8 / 4 = 2. GCC de uyarır (kopyada denendi):

GERÇEK ÇIKTI

```
warning: ‘sizeof’ on array function parameter ‘a’ will return size of ‘const int *’ [-Wsizeof-array-argument]
$ ./q02
6
fonksiyonda: 2
95
```

S3 · A 3 üç kez, 6 iki kez, 4 hiç: **değer = indis**

GERÇEK ÇIKTI

```
$ ./q03
3 2 0
```

BÜYÜK SORU

Haftanın Büyük Sorusu

Bir fotoğraf, bir satranç tahtası ya da bir sınıf listesi bilgisayar için nedir ve aynı matrisi satır satır ya da sütun sütun dolaşmak neden farklı hızda sonuç verir?

Tablo bellekte nasıl durur?

Bellek tek boyutlu bir bayt dizisi. Izgara nerede?

m[i][j] nerede?

Derleyici bir hücrenin adresini nasıl hesaplar?

Sıra hızı değiştirir mi?

Aynı toplam, iki dolaşma sırası: ölçeceğiz.

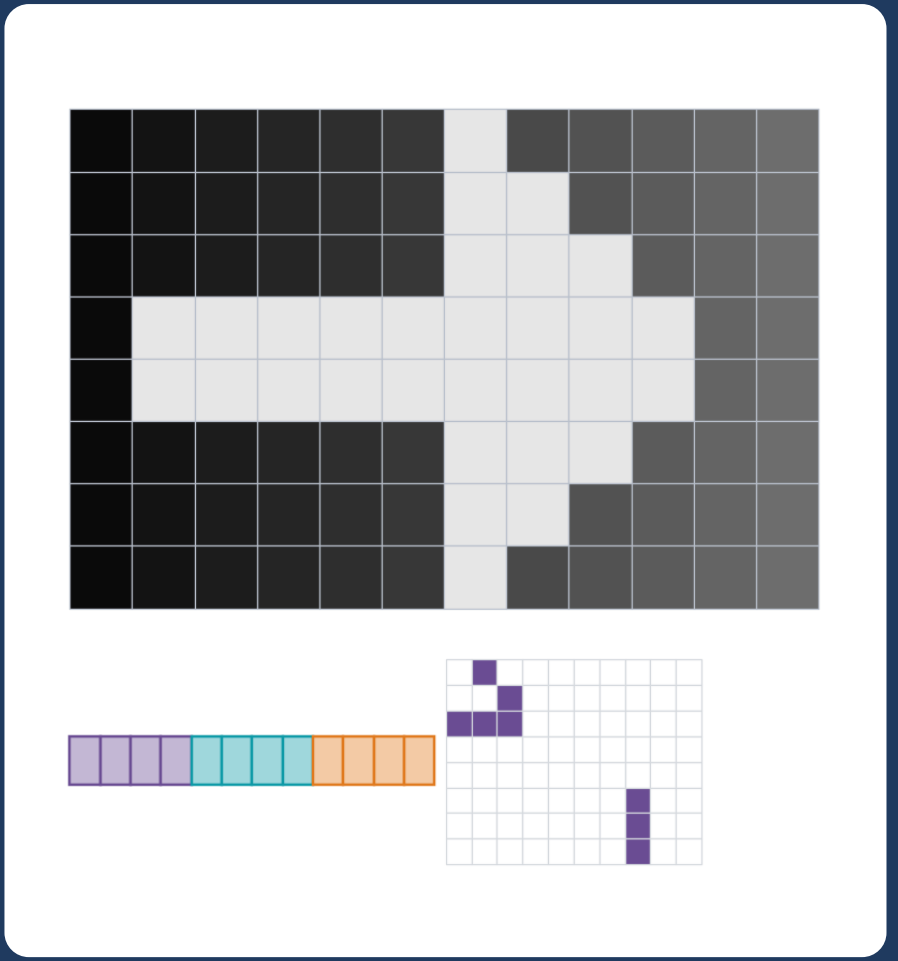
*Haftanın sonunda üç soruyu da **gerçek adresler ve ölçümlerle** yanıtlayacağız.*

GERÇEK DÜNYADAN

1970

Conway’in Hayat Oyunu

- Ekim 1970: **Martin Gardner**, Scientific American dergisinin “Mathematical Games” köşesi
- Kareli düzlem; her hücre **canlı** ya da **ölü**; sekiz komşu
- **Doğum:** tam 3 canlı komşu · **Yaşam:** 2 ya da 3 · aksi hâlde **ölüm** (B3/S23)
- **Planör** (glider): dört nesilde bir hücre çapraz kayar
- Program olarak: bir **2B dizi** + **iç içe döngüler** (Örnek 13)



Gardner (1970), Scientific American 223(4), 120–123 · görsel: H11_11 ve H11_13’ün gerçek çıktılarından

Bir fotoğraf = bir sayı matrisi

Her filtre okuduğu matristen AYRI bir matrise yazar



8 × 12 gri tonlu piksel matrisi (0 siyah … 255 beyaz); değerler H11_11'in gerçek çıktısından

- Gri tonlu görüntüde her piksel bir sayı: **0 siyah … 255 beyaz** (1 bayt); 1920 × 1080 bir kare **2 073 600** piksel
- “Negatif”, “siyah-beyaz”, “ayna”, “bulanıklaştır” düğmeleri: matris üzerinde **iç içe döngüler** (Örnek 11)

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 11.1	İki boyutlu dizileri bildirir, satır–sütun indisleriyle erişir ve iç içe döngülerle işler.
ÖÇ 11.2	Çok boyutlu dizilerin satır öncelikli (row-major) yerleşimini açıklar ve eleman adresini hesaplar.
ÖÇ 11.3	Matris işlemlerini (toplama, devrik, çarpma, köşegen, satır/sütun toplamları) gerçekler.
ÖÇ 11.4	Izgara tabanlı problemleri (oyun tahtası, koltuk planı, görüntü) iki boyutlu dizilerle modeller; komşuluk ve sınır denetimlerini doğru uygular.

01

İki boyutlu diziler ve bellek

Bildirim, erişim, satır öncelikli şerit, fonksiyona aktarım, komşuluk ve önbellek

KAVRAM

Neden iki boyut? Tablolar ve ızgaralar

Not tablosu

- satır = öğrenci, sütun = ders
- `int notlar[5][4];`
- `notlar[2][1]`: 3. öğrencinin 2. dersi

Sinema salonu

- satır = sıra, sütun = koltuk
- `int salon[5][8];`
- bilet “3. sıra 4. koltuk” = `salon[2][3]`

Oyun tahtası

- tic-tac-toe, satranç, mayın tarlası
- `char t[3][3];`
- hücre ve komşuları

Görüntü

- satır = piksel satırı (y), sütun = x
- `int g[YUK][GEN];`
- 0 siyah … 255 beyaz

Bildirim ve ilk değer

H11_K01_ilk_deger.c · satır 26–34

```c
26      int a[2][3] = {{1, 2, 3}, {4, 5, 6}};     /* satır satır */
27      int c[2][3] = {{1}, {4, 5}};              /* kısmi: kalanlar 0 */
28      int d[2][3] = {0};                        /* hepsi 0 */
29      int e[][3] = {{1, 2, 3}, {4, 5, 6}, {7}}; /* satır sayısı: 3 */
30
31      yazdir('a', 2, a);
32      yazdir('c', 2, c);
33      yazdir('d', 2, d);
34      yazdir('e', 3, e);
```

GERÇEK ÇIKTI

```
$ ./ilk_deger
a:  { 1 2 3 }  { 4 5 6 }
c:  { 1 0 0 }  { 4 5 0 }
d:  { 0 0 0 }  { 0 0 0 }
e:  { 1 2 3 }  { 4 5 6 }  { 7 0 0 }
sizeof a = 24, sizeof a[0] = 12, sizeof a[0][0] = 4
satır sayısı = 3, sütun sayısı = 3
```

- İlk değer **satır satır**; eksikler **0**
- Liste varsa **yalnızca ilk boyut** boş kalabilir
- `sizeof a[0]` = **bir satır** (12 bayt)
- Satır sayısı: `sizeof e / sizeof e[0]`

Erişim ve iç içe döngü: Örnek 1

H11_01_matris_oku_yaz.c · satır 19–27

```c
19      for (int i = 0; i < SATIR; i++) {          /* dış döngü: satır */
20          for (int j = 0; j < SUTUN; j++) {      /* iç döngü: sütun */
21              if (scanf("%d", &m[i][j]) != 1) {
22                  printf("Geçersiz girdi: m[%d][%d] okunamadı.\n",
23                         i, j);
24                  return 1;
25              }
26          }
27      }
```

H11_01_matris_oku_yaz.c · satır 30–35

```c
30      for (int i = 0; i < SATIR; i++) {
31          for (int j = 0; j < SUTUN; j++) {
32              printf("%5d", m[i][j]);            /* 5 karakterlik alan */
33          }
34          printf("\n");                          /* satır bitti */
35      }
```

- Dış döngü **satır** (`i`), iç döngü **sütun** (`j`); `&m[i][j]` ile `scanf`, dönüş değeri denetlenir

GERÇEK ÇIKTI

```
$ ./matris_oku_yaz
3 x 3 matrisi satır satır girin:
4 -7 12
0 105 3
-18 6 9
Matris:
    4   -7   12
    0  105    3
  -18    6    9

$ ./matris_oku_yaz
3 x 3 matrisi satır satır girin:
1 2 x
Geçersiz girdi: m[0][2] okunamadı.
```

- `%5d`: sütunlar hizalı
- 2. çalıştırma: `x` → hata

TAHMİN ET · KISMİ İLK DEĞER

Program ne yazdırır?

quiz/H11_Q04_kismi_ilk_deger.c · satır 8–13

```
 8  int main(void)
 9  {
10      int m[3][3] = {{1}, {2, 3}, {4, 5, 6}};
11      printf("%d %d %d\n", m[0][1], m[1][1], m[2][0]);
12      return 0;
13  }
```

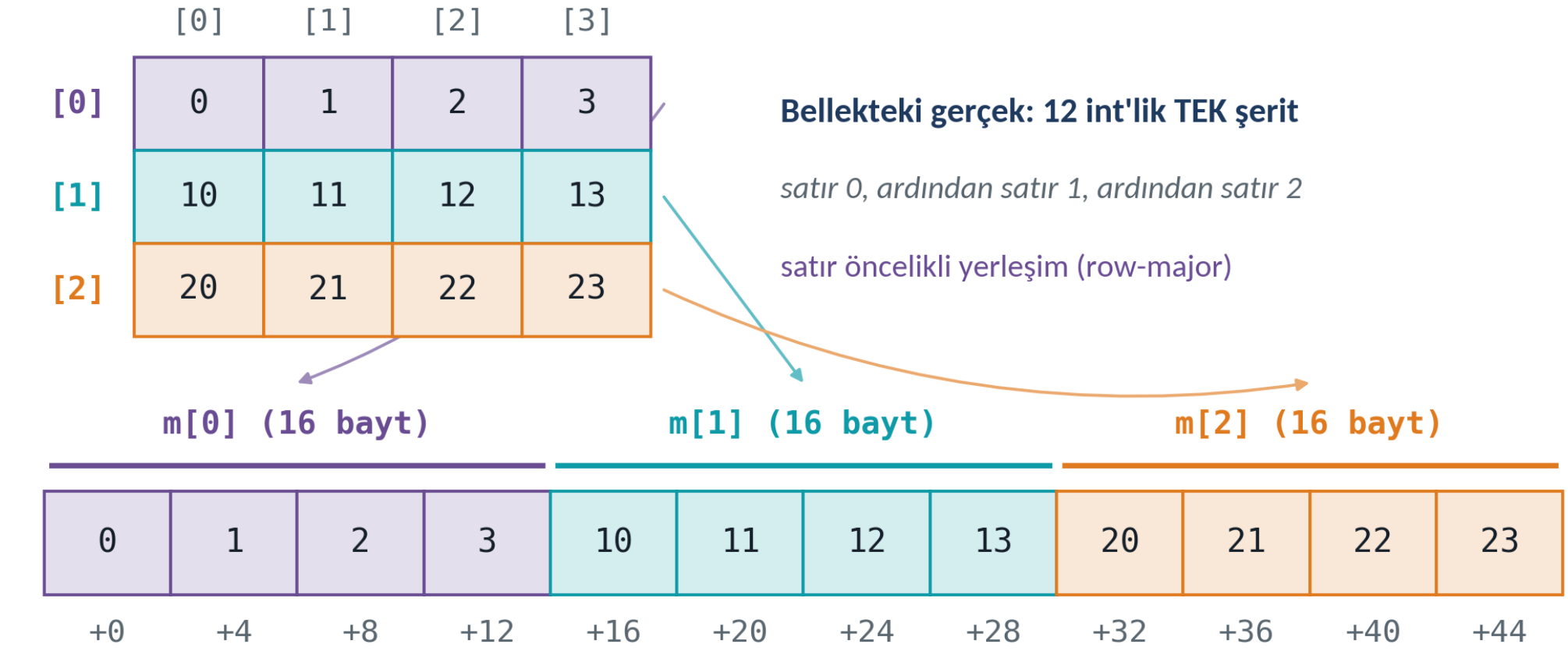
A 1 3 4

B 0 3 4

C 0 2 4

D 0 3 5

Kaputun altında: ızgara bellekte tek şerittir



&m[0][0] = 0x7ffd9dd7e190 (gerçek çalıştırma) · alttaki sayılar: &m[0][0]'dan bayt uzaklığı

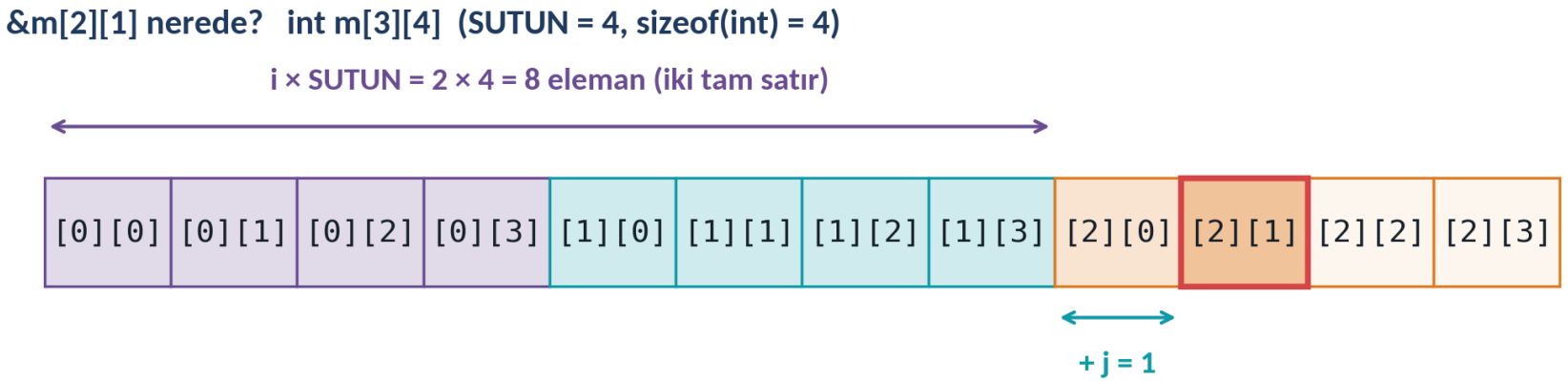
Gerçek adresler: H11_K02

GERÇEK ÇIKTI

```
$ ./bellek_serit      # 1. çalıştırma (kesit)
eleman   adres            uzaklık  (i*SUTUN+j)*4
m[0][0]  0x7ffd9dd7e190     0         0
m[0][1]  0x7ffd9dd7e194     4         4
m[0][2]  0x7ffd9dd7e198     8         8
m[0][3]  0x7ffd9dd7e19c    12        12
m[1][0]  0x7ffd9dd7e1a0    16        16
m[1][1]  0x7ffd9dd7e1a4    20        20
m[1][2]  0x7ffd9dd7e1a8    24        24
m[1][3]  0x7ffd9dd7e1ac    28        28
m[2][0]  0x7ffd9dd7e1b0    32        32
…
&m[1][0] - &m[0][3] = 4 bayt (1 eleman)
&m[1][0] - &m[0][2] = 8 bayt (2 eleman)
sizeof m = 48, sizeof m[1] = 16 (bir satır)
m[1] = 0x7ffd9dd7e1a0 = &m[1][0] (satırın adresi)
```

- Her adım **+4 bayt**; satır sınırında da
- `&m[1][0] – &m[0][3]` = **4 bayt**
- Ölçülen uzaklık = **(i × SUTUN + j) × 4**
- `m[1]`: bir satırın **adresi** (H12)
- 2. çalıştırma: `&m[0][0]` = 0x7ffe123f5950 (ASLR); uzaklıklar **aynı**
- `(char *)` farkı yalnızca bayt uzaklığını yazdırmanın yolu; ayrıntısı **H12**'de

Adres formülü



adres = taban + (i × SUTUN + j) × sizeof(int)

= taban + (2 × 4 + 1) × 4 = taban + 36 bayt

- Önce **i tam satır** atla: i × SUTUN eleman
- Sonra satır içinde **j** eleman
- Formülde **SATIR yok**, **SUTUN var**
- Bu yüzden fonksiyon sütun sayısını bilmek zorunda

TAHMİN ET · SATIR ÖNCELİKLİ ADRES

`int m[3][4]` için `&m[2][1]`, `&m[0][0]`'dan kaç bayt uzaktadır?

quiz/H11_Q05_uzaklik.c · satır 8–13

```
 8  int main(void)
 9  {
10      int m[3][4];
11      printf("%td\n", (char *)&m[2][1] - (char *)&m[0][0]);
12      return 0;
13  }
```

A 9

B 24

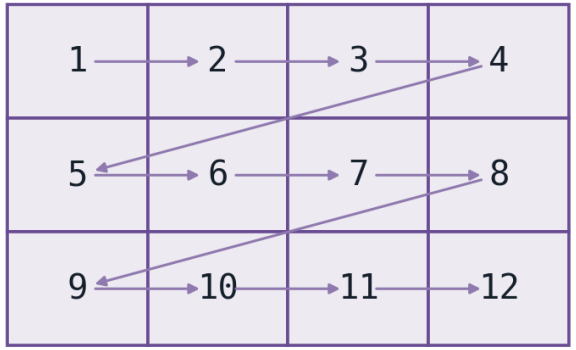
C 36

D 28

Not: `(char *)` dönüşümü ve iki adresin farkı (`%td`) burada yalnızca bayt uzaklığını yazdırmanın yoludur; ayrıntısı H12'de (işaretçiler).

Dolaşma sırası: satır satır mı, sütun sütun mu?

Satır öncelikli: i dışta, j içte

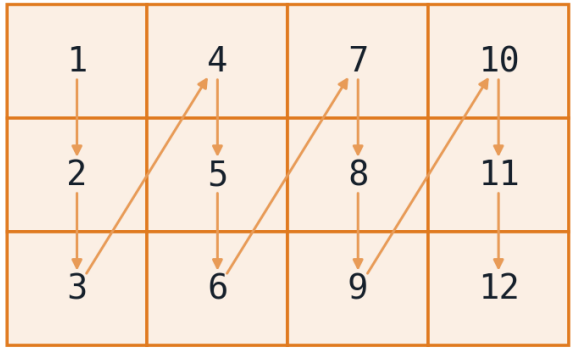


ziyaret sırası (hücredeki sayı)

adım +4 bayt (komşu)

`uzaklık: 0 4 8 12 16 20 …`

Sütun öncelikli: j dışta, i içte



ziyaret sırası (hücredeki sayı)

adım +16 bayt (bir satır)

`uzaklık: 0 16 32 4 20 36 …`

- Satır öncelikli: `i` dışta, `j` içte → adım **+4**
- Sütun öncelikli: döngüler **yer değiştirir** → adım **+16** (bir satır boyu)
- Toplam aynı; **süre** aynı mı? (Örnek 12)

GERÇEK ÇIKTI · ZİYARET EDİLEN ELEMANIN &M[0][0]'DAN BAYT UZAKLIĞI

```
$ ./dolasma_sirasi
satır öncelikli: 0 4 8 12 16 20 24 28 32 36 40 44
sütun öncelikli: 0 16 32 4 20 36 8 24 40 12 28 44
```

İki boyutlu diziyi fonksiyona aktarmak

`void yazdir(int m[][4], int n)`

SUTUN = 4 biliniyor ✓

`m[i][j] → taban + (i·4 + j)·4`

fonksiyona yalnızca

dizinin ADRESİ gider (H10)

`void yazdir(int m[][], int n)`

SUTUN bilinmiyor ✗

`m[i][j] → taban + (i·? + j)·4`

`error: array type has`

`incomplete element type`

İlk boyut boş kalabilir (satır sayısı ayrı parametre); diğer TÜM boyutlar yazılmalı

- Dizi yine **adresiyle** gider (H10)
- İlk boyut boş kalabilir: satır sayısı **ayrı parametre**
- Sütun sayısı **tipte zorunlu**: `int m[][SUTUN]`

SÜTUN BOYUTU YOK: DERLEME HATASI (HA3, KASITLI; UZUN SATIR KAYDIRILDI)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H11_HA3_sutun_boyutu_yok.c -o ha3
H11_HA3_sutun_boyutu_yok.c:10:17: error: array type has incomplete element type ‘int[]’
H11_HA3_sutun_boyutu_yok.c:10:17: note: declaration of ‘m’ as multidimensional array must
    have bounds for all dimensions except the first
```

Bir satır 1B dizidir, bir sütun değildir

H11_K03_satir_fonksiyona.c · satır 24–29

```c
void sutun_kopyala(int m[][SUTUN], int satir, int j, int hedef[])
{
    for (int i = 0; i < satir; i++) {
        hedef[i] = m[i][j];                    /* sütun → 1B dizi */
    }
}
```

```
$ ./satir_fonksiyona
0. satırın toplamı: 10
1. satırın toplamı: 26
2. satırın toplamı: 42
2. sütunun toplamı (kopya ile): 21
```

H11_K03_satir_fonksiyona.c · satır 36–40

```c
    for (int i = 0; i < SATIR; i++) {
        printf("%d. satırın toplamı: %d\n", i, dizi_toplam(m[i], SUTUN));
    }
    sutun_kopyala(m, SATIR, 2, sutun);
    printf("2. sütunun toplamı (kopya ile): %d\n", dizi_toplam(sutun, SATIR));
```

- `m[i]` bitişik bir satır: `dizi_toplam(m[i], SUTUN)` doğrudan · sütun bitişik değil: önce **kopyala**
- Mini-OBS v0.11'in `sutun_kopyala` fonksiyonu aynı desendir

MERAK KUTUSU

const 2B parametre: C17'nin pürüzü, C23'ün düzeltmesi

H11_K04_const_parametre.c · satır 16–23

```
16  double iz(const double m[][N])
17  {
18      double toplam = 0.0;
19      for (int i = 0; i < N; i++) {
20          toplam += m[i][i];
21      }
22      return toplam;
23  }
```

- const dizi → const parametre: **sorunsuz**
- const olmayan dizi: C17'de **uyumsuz** sayılır
- C23 düzeltti (WG14 N2607)
- Değişken boyut: **VLA** parametresi `m[satir][sutun]` (H11_K08; MSVC'de yok)

KOPYADA DİZİNİN CONST'U SİLİNİNCE (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H11_K04_const_parametre.c -o const_parametre
warning: invalid use of pointers to arrays with different qualifiers in ISO C before C2X [-Wpedantic]
$ gcc -std=c2x -Wall -Wextra -pedantic -g H11_K04_const_parametre.c -o const_parametre
(ileti yok)
$ clang -std=c17 -Wall -Wextra -pedantic -g H11_K04_const_parametre.c
(ileti yok)
```

TAHMİN ET · 2B DİZİ PARAMETRESİ (main'e göre imzayı seçin)

Hangi imza derlenir?

quiz/H11_Q07_parametre.c · satır 17–22

```
17  int main(void)
18  {
19      int a[2][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}};
20      yazdir(a, 2);
21      return 0;
22  }
```

A `void yazdir(int m[][], int satir)`

B `void yazdir(int m[2][], int satir)`

C `void yazdir(int m[][], int satir, int sutun)`

D `void yazdir(int m[][4], int satir)`

Komşuluk ve sınır denetimi

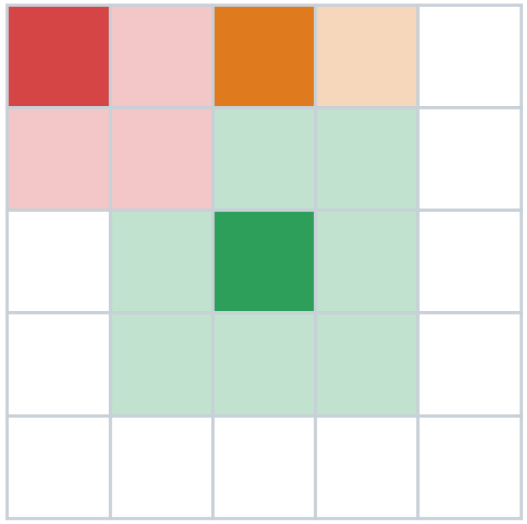
3 × 3 pencere: (di, dj)

-1,-1	-1,0	-1,+1
0,-1	(i,j)	0,+1
+1,-1	+1,0	+1,+1

■ 4-komşu: dik yönler

■ + çaprazlar = 8-komşu

5 × 5 ızgarada geçerli komşular



köşe (0,0)

4-komşu 2 · 8-komşu 3

kenar (0,2)

4-komşu 3 · 8-komşu 5

iç (2,2)

4-komşu 4 · 8-komşu 8

dışarıdaki komşu YOK

sayılır (sınır denetimi)

- **4-komşu**: dik yönler; **8-komşu**: + çaprazlar
- Köşe 3, kenar 5, iç hücre **8** komşu (H11_K07)
- Erişimden **önce**: `0 <= ni < SATIR`, `0 <= nj < SUTUN`
- `&&` kısa devresi dışarıyı hiç okumaz

Üç boyutlu dizilere kısa bakış

`notlar[ŞUBE][ÖĞRENCİ][DERS] = [2][3][2]`

[0] A şubesi

70	80
55	65
90	85

[1] B şubesi

60	75
88	92
45	50

katman = şube, satır = öğrenci, sütun = ders

Bellekte yine tek şerit (12 int):



A şubesi (24 bayt) B şubesi

```
uzaklık(notlar[s][o][d]) =
((s·ÖĞR + o)·DERS + d)·4
[1][2][1]: ((1·3+2)·2+1)·4 = 44
```

- `notlar[ŞUBE][ÖĞRENCİ][DERS]`
- Üç iç içe döngü
- Bellekte yine **tek şerit**
- Renkli görüntü: `[YUK][GEN][3]`

Kaputun altında: önbellek satırı

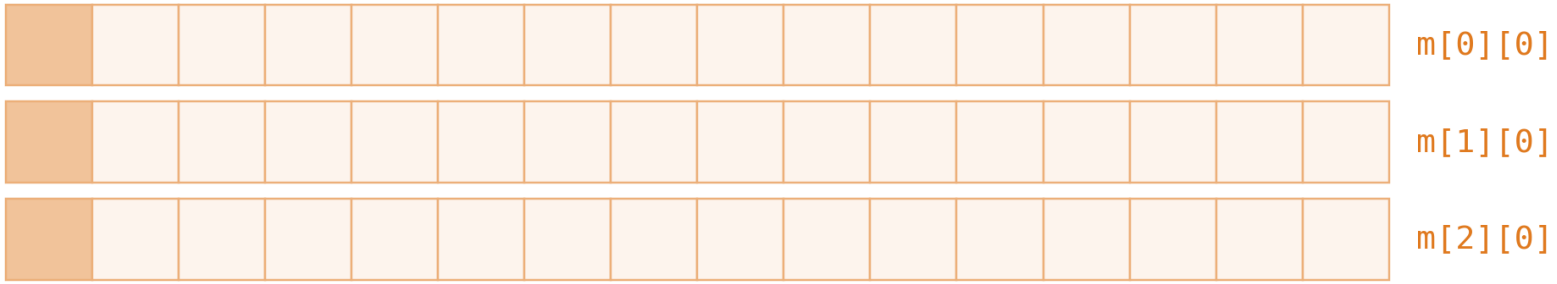
Önbellek satırı = 64 bayt = 16 int (getconf LEVEL1_DCACHE_LINESIZE → 64)

Satır öncelikli: bir satır getirilir, 16 erişimin 16'sı kullanılır



m[i][0] ... m[i][15]: tek satır, hepsi işe yarar

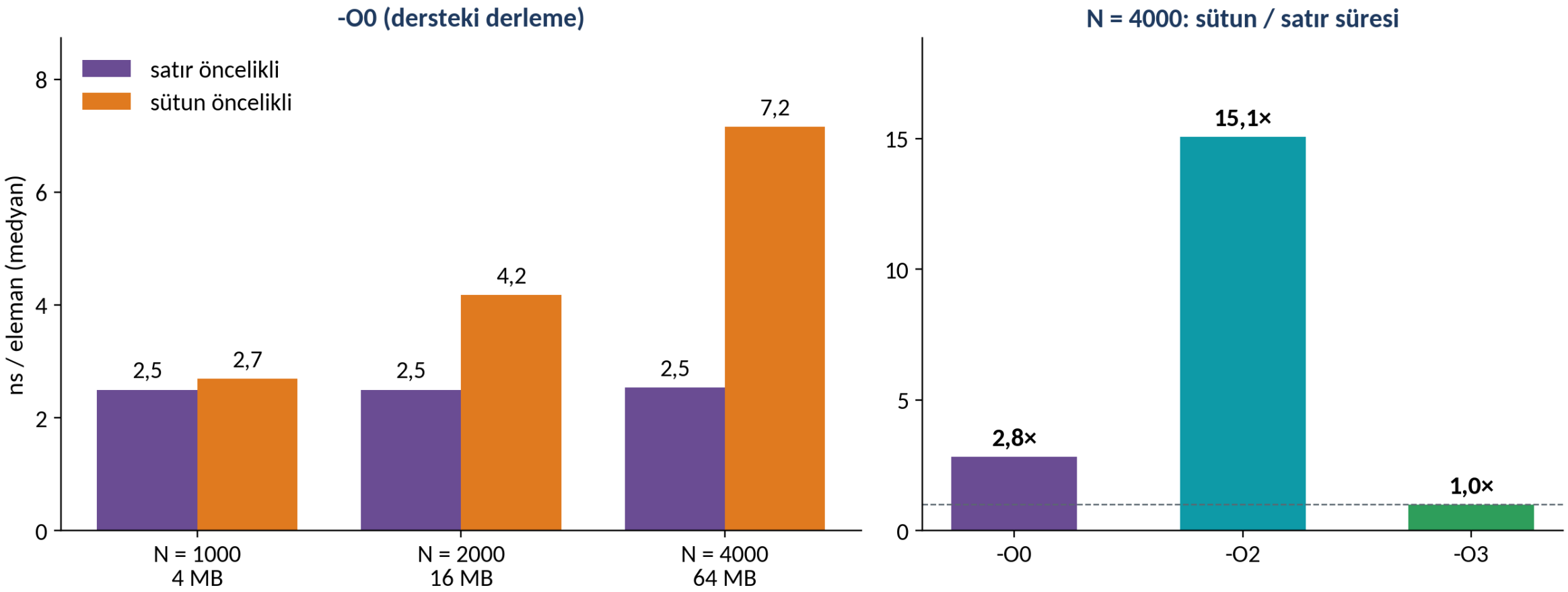
Sütun öncelikli (N = 4000): her erişim 16 000 bayt ötede, yeni bir satır



64 bayt getirilir, yalnızca 4'ü kullanılır (1/16)

- Önbellek satırı: **64 bayt** = 16 int (getconf)
- Satır öncelikli: **16 erişime 1 getirme**
- Sütun öncelikli (N = 4000): her erişim **16 000 bayt** ötede
- 4000 × 4000 int = **64 MB** > L3 (33 MiB)

Örnek 12: satır ve sütun öncelikli dolaşma süre deneyi



Derleyici ne yaptı? -O2 vektörleştirdi, -O3 döngüleri değiştirdi

-FOPT-İNFO-LOOP (GERÇEK)

```
$ gcc -std=c17 -O2 -fopt-info-loop H11_12_onbellek_sure.c -o /dev/null
H11_12_onbellek_sure.c:26:27: optimized: loop vectorized using 16 byte vectors
$ gcc -std=c17 -O3 -fopt-info-loop H11_12_onbellek_sure.c -o /dev/null
H11_12_onbellek_sure.c:36:23: optimized: loops interchanged in loop nest
H11_12_onbellek_sure.c:37:27: optimized: loop vectorized using 16 byte vectors
$ gcc -Q --help=optimizers -O2 | grep floop-interchange
-floop-interchange                          [disabled]
$ gcc -Q --help=optimizers -O3 | grep floop-interchange
-floop-interchange                          [enabled]
```

2,8×

-O0: sütun / satır

15,1×

-O2: satır döngüsü vektörleştirildi

1,0×

-O3: “loops interchanged”

02

Örnek merdiveni

Toplamadan Hayat Oyunu’na: on dört örnek, gerçek çıktılarla

Örnek merdiveni: her basamak bir yeni öge

#	Örnek	Yeni öge	Düzey
1	3 × 3 matris oku–yazdır (hizalı)	2B bildirim ve iç içe döngü	★
2	Satır ve sütun toplamları	Dolaşma yönü	★
3	Matris toplama	Eleman eleman işlem	★
4	Devrik (transpoz) matris	İndislerin yer değiştirmesi	★★
5	Köşegen toplamı, birim ve simetrik matris denetimi	Koşullu dolaşma	★★
6	Öğrenci × ders not tablosu	Satır/sütun ortalamaları; gerçek veri modeli	★★
7	Sinema salonu koltuk rezervasyonu	Durum matrisi; kullanıcı etkileşimi	★★
8	Matris çarpımı	Üçlü iç içe döngü; n^3 işlem sezgisi	★★★
9	Tic-tac-toe ve kazanan denetimi	Satır, sütun ve köşegen denetimi	★★★
10	Mayın tarlası komşu sayıları	8-komşuluk ve sınır denetimi	★★★
11	Görüntü işleme: negatif, eşik, ayna, bulanıklaştırma	Piksel matrisi; ASCII sanat çıktısı	★★★
12	Satır ve sütun öncelikli dolaşma süre deneyi	Önbellek etkisi	★★★
13	Conway’in Hayat Oyunu (meydan okuma)	Nesil hesaplama; iki tampon matris	★★★
14	Spiral matris yazdırma (meydan okuma)	Sınır yönetimi	★★★

Örnek 2 ve 3: dolaşma yönü ve eleman eleman işlem

H11_02_satir_sutun_toplam.c · satır 35–43

```c
35      for (int j = 0; j < GUN; j++) {          /* dış döngü: SÜTUN */
36          int sutun_toplami = 0;
37          for (int i = 0; i < KASA; i++) {     /* iç döngü: satır */
38              sutun_toplami += satis[i][j];
39          }
40          printf("%5d", sutun_toplami);
41          genel += sutun_toplami;
42      }
43      printf(" | %6d\n", genel);
```

ÖRNEK 2 · GERÇEK ÇIKTI

```
$ ./satir_sutun_toplam
         Pzt   Sal   Çar   Per   Cum | Toplam
Kasa 1    12    15     9    20    18 |     74
Kasa 2     7    11    14    10    16 |     58
Kasa 3    21    19    17    25    22 |    104
Toplam    40    45    40    55    56 |    236
```

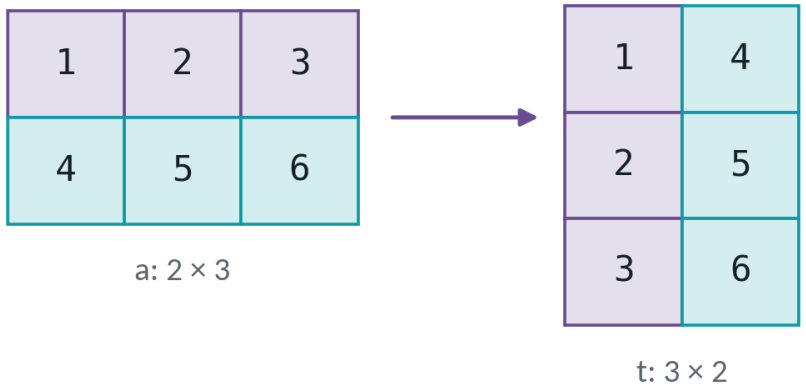
Örnek 3: aynı konumdaki elemanlar, `c[i][j] = a[i][j] + b[i][j]`; boyutlar aynı olmalı, `c = a + b;` yazılamaz

ÖRNEK 3 · GERÇEK ÇIKTI

```
$ ./matris_toplama
  1  2  3       10  20  30       11  22  33
  4  5  6   +   40  50  60   =   44  55  66
```

Örnek 4: devrik matris

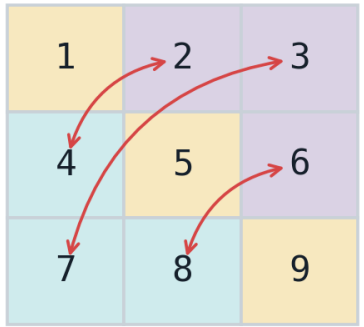
Yeni matrise: t[j][i] = a[i][j]



satırlar sütun olur:

boyutlar yer değiştirir

Yerinde (yalnızca kare): j = i + 1



3 takas

= N(N−1)/2

köşegen

yerinde

j = 0'dan başlarsa her çift

İKİ kez takas edilir: değişmez!

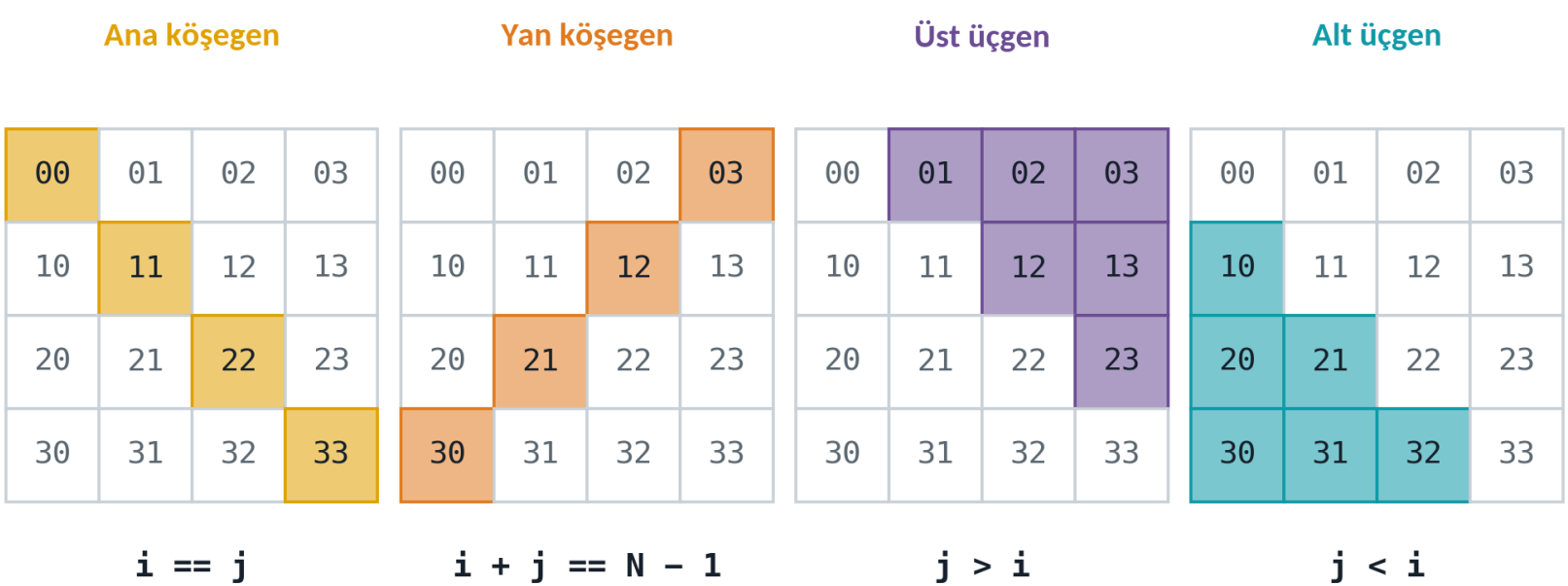
GERÇEK ÇIKTI (KESİT)

```
Yerinde devrik (3 x 3):
  1  4  7
  2  5  8
  3  6  9
Takas sayısı: 3 (N(N-1)/2)
```

H11_04_devrik.c · satır 36–40

```
36          for (int j = i + 1; j < N; j++) {     /* yalnızca köşegenin üstü */
37              int gecici = k[i][j];
38              k[i][j] = k[j][i];
39              k[j][i] = gecici;
40              takas++;
```

Örnek 5: köşegenler, birim ve simetrik matris



hücredeki iki rakam: i ve j (N = 4)

- Ana: `i == j`; yan: `i + j == N − 1`
- Üst üçgen `j > i`, alt `j < i`
- Simetri: yalnızca üst üçgen (`j = i + 1`)
- İlk aykırılıkta `return false`

GERÇEK ÇIKTI

```
$ ./kosegen_simetri
   Ana  Yan  Birim    Simetrik   Üst üçgen
A   10   34  hayır     evet       hayır
B    4    0  evet      evet       evet
C   17   10  hayır     hayır      evet
D   34   34  hayır     hayır      hayır
```

Beş denetim, beş fonksiyon: `ana_kosegen`, `yan_kosegen`, `birim_mi`, `simetrik_mi`, `ust_ucgen_mi`. Hepsinin parametresi `int m[][N]`: 2B diziyi fonksiyona aktarmanın ilk kullanımı.

Örnek 6: öğrenci × ders not tablosu

H11_06_not_tablosu.c · satır 37–44

```
37      printf("Ort.    ");
38      for (int j = 0; j < DERS; j++) {          /* sütun: bir ders */
39          int toplam = 0;
40          for (int i = 0; i < OGRENCI; i++) {
41              toplam += notlar[i][j];
42          }
43          printf(" %5.1f", (double)toplam / OGRENCI);
44      }
```

GERÇEK ÇIKTI

```
$ ./not_tablosu
Öğrenci   MAT   FİZ   KİM   PRG    Ort.
      1    78    85    62    90   78.75
      2    55    70    48    66   59.75
      3    92    88    95    81   89.00
      4    67    59    73    70   67.25
      5    84    91    77    87   84.75
Ort.     75.2  78.6  71.0  78.8
En yük. 92(3) 91(5) 95(3) 90(1)
Genel ortalama: 75.90
```

- Satır ortalaması = **öğrenci**, sütun ortalaması = **ders**
- Sütun ortalaması: **j dışta, i içte** (Örnek 2'nin deseni)
- Elle denetim: 3. öğrenci (92 + 88 + 95 + 81) / 4 = **89,00**
- En yüksek notun **sahibi**: izleme tablosu

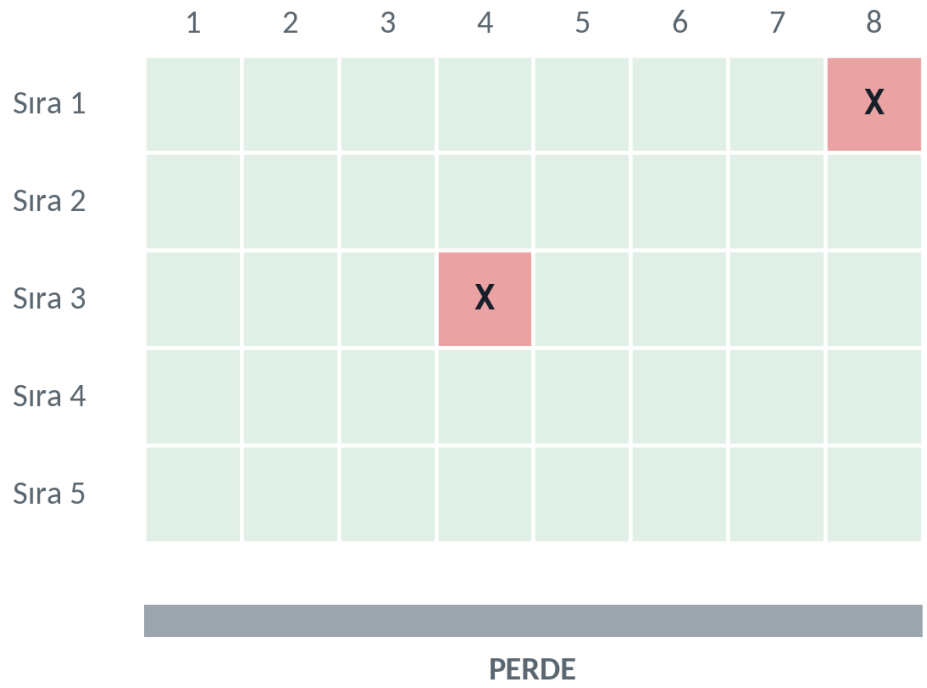
İzleme: MAT sütununda (j = 0) en yüksek not

H11_06_not_tablosu.c · satır 47–53

```
47          int enb = 0;                          /* en yüksek notun satırı */
48          for (int i = 1; i < OGRENCI; i++) {
49              if (notlar[i][j] > notlar[enb][j]) {
50                  enb = i;
51              }
52          }
53          printf(" %2d(%d)", notlar[enb][j], enb + 1);
```

i	notlar[i][0]	notlar[enb][0]	büyük mü?	enb
başl.	–	–	–	0
1	55	78	hayır	0
2	**92**	**78**	**evet**	**2**
3	67	92	hayır	2
4	84	92	hayır	2

Örnek 7: sinema salonu koltuk rezervasyonu



Kullanıcı: sıra 3, koltuk 4

```
salon[3 - 1][4 - 1]
  = salon[2][3]
```

Durum matrisi: 0 boş, 1 dolu

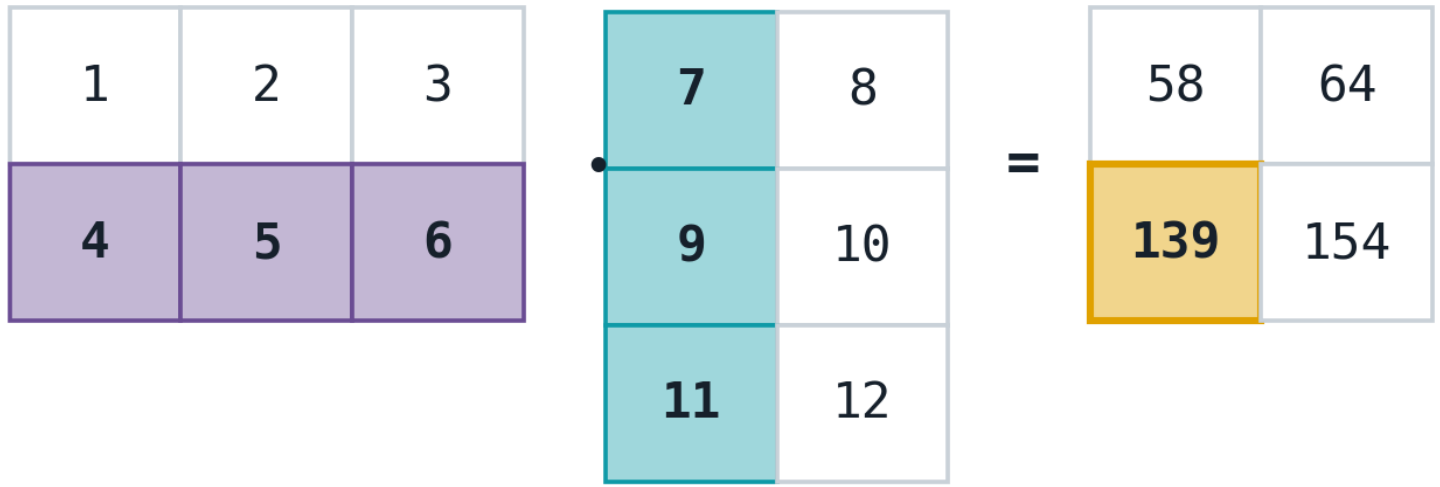
Reddedilir:

- aralık dışı (6 2)
- dolu koltuk (3 4 ikinci kez)

Dolu: 2 / 40 (gerçek çalıştırma)

- **Durum matrisi**: 0 boş, 1 dolu
- Kullanıcı **1'den**, C **0'dan** sayar: `salon[s - 1][k - 1]`
- Aralık denetimi erişimden **önce**
- Reddedilir: aralık dışı (6 2), dolu koltuk (3 4)

Örnek 8: matris çarpımı = satır × sütun



`c[1][0]` =

A'nın 1. satırı ·

B'nin 0. sütunu

= 4·7 + 5·9 + 6·11

= 139

hücre başına 3 çarpma

toplam 2·2·3 = 12

```
c[i][j] = 0;  for k:  c[i][j] += a[i][k] * b[k][j];
```

Örnek 8: kod ve n³ sayacı

H11_08_matris_carpim.c · satır 14–28

```
14  long long carp(int n, int m, int p, int a[][AZAMI], int b[][AZAMI],
15                 int c[][AZAMI])
16  {
17      long long carpma = 0;                    /* yapılan çarpma sayısı */
18      for (int i = 0; i < n; i++) {            /* a'nın satırı */
19          for (int j = 0; j < p; j++) {        /* b'nin sütunu */
20              c[i][j] = 0;                     /* ÖNCE sıfırla */
21              for (int k = 0; k < m; k++) {
22                  c[i][j] += a[i][k] * b[k][j];
23                  carpma++;
24              }
25          }
26      }
27      return carpma;
28  }
```

```
$ ./matris_carpim
(2 × 3) · (3 × 2) = 2 × 2:
    58    64
   139   154
çarpma sayısı: 12 (2 · 3 · 2)

     n      çarpma          n³
    10        1000        1000
    20        8000        8000
    40       64000       64000
    80      512000      512000
   160     4096000     4096000
```

- n iki katı → çarpma **8 katı** (n^3)
- `c[i][j] = 0;` unutulursa: HA4

TAHMİN ET · MATRİS ÇARPIMI

(2 × 3) · (3 × 4) çarpımında program ne yazdırır?

quiz/H11_Q08_carpma_sayisi.c · satır 12–23

```
12      int c[2][4];
13      int sayac = 0;
14      for (int i = 0; i < 2; i++) {
15          for (int j = 0; j < 4; j++) {
16              c[i][j] = 0;
17              for (int k = 0; k < 3; k++) {
18                  c[i][j] += a[i][k] * b[k][j];
19                  sayac++;
20              }
21          }
22      }
23      printf("%d %d\n", sayac, c[1][3]);
```

A 24 15

B 12 15

C 24 6

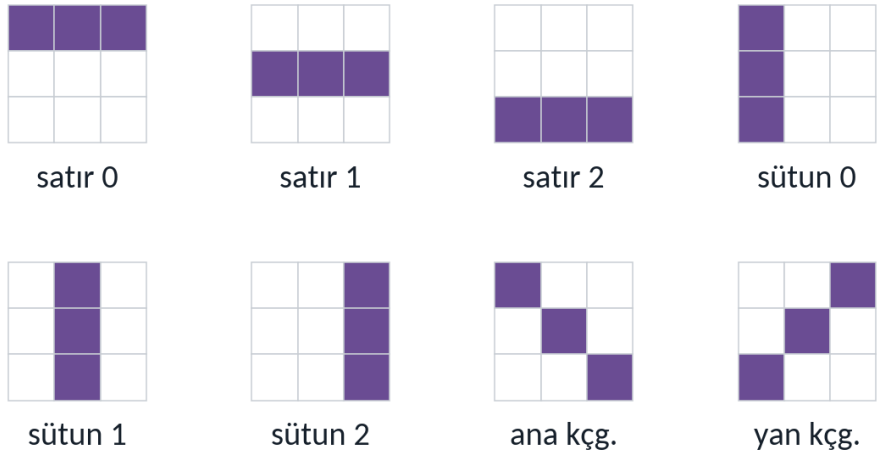
D 36 15

Örnek 9: tic-tac-toe ve kazanan denetimi

Son tahta (1. çalıştırma)

0		X
0	X	
X		

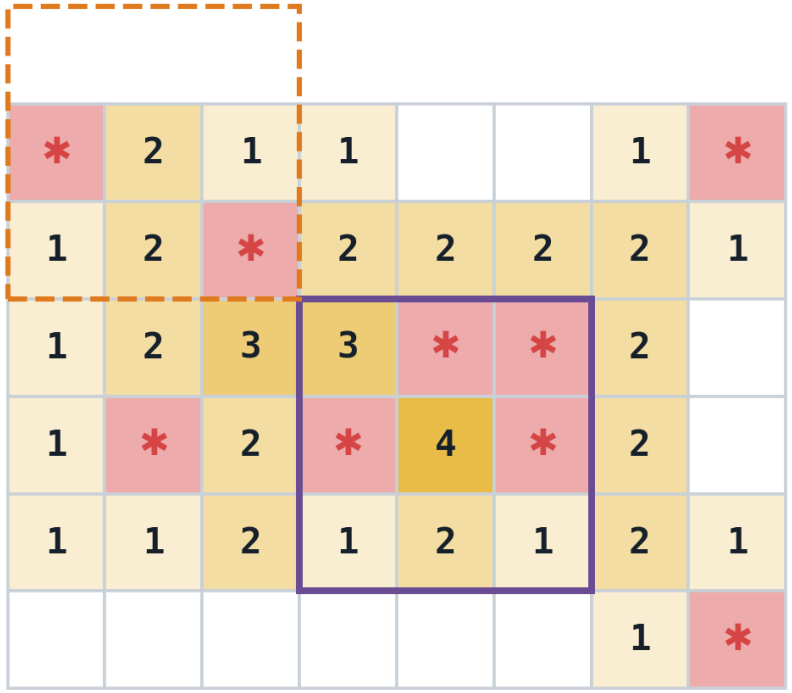
Denetlenen 8 çizgi



- 3 satır + 3 sütun + 2 köşegen = **8 çizgi**
- Tek döngüde: `t[i][0..2]` ve `t[0..2][i]`
- Geçersiz hamle: `continue` ile aynı oyuncu
- 1. çalıştırma: **X 5 hamlede** (yan köşegen)

Örnek 10: mayın tarlası komşu sayıları

H11_10 çıktısı: sayı = 8 komşudaki mayın sayısı



■ **(3,4): 4 komşu mayın**

pencerenin tamamı içeride

■ **(0,1) kenarda:**

pencerenin üst satırı dışarıda;

m[-1][...] okunmamalı

```
if (ni >= 0 && ni < SATIR &&
    nj >= 0 && nj < SUTUN)
```

- 3 × 3 pencere: `di, dj ∈ {-1, 0, 1}`
- `(0, 0)` hücrenin kendisi: atla
- Kenarda pencere **dışarı taşar**
- Erişimden önce `if (ni >= 0 && …)`

Örnek 11: görüntü işleme ve bulanıklaştırma

Özgün (satır 0–3, sütun 4–7)

46	55	230	73
46	55	230	230
46	55	230	230
230	230	230	230

keskin kenar: 55 → 230

Merkez (1,5): 9 pikselin ortalaması

```
(46 + 55 + 230 + 46 + 55
 + 230 + 46 + 55 + 230) / 9
= 993 / 9 ≈ 110   (çıktıdaki değer: 110)
```

Köşe (0,0): yalnızca 4 piksel → 15

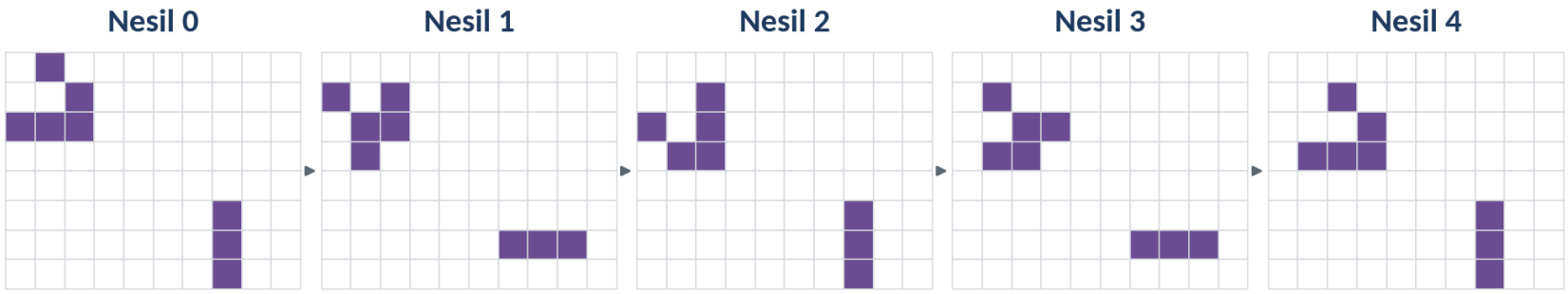
Kenar (0,5): 6 piksel → 110

Kenar politikası: yalnızca görüntünün içindeki komşuların ortalaması (adet 4, 6 ya da 9)

- Negatif `255 - p`, eşik `p >= 128`, ayna `g[i][GEN-1-j]`
- Bulanık: 3 × 3 komşuluğun **ortalaması**
- Kenar politikası: yalnızca **içerideki** komşular (4, 6, 9)
- Sonuç **ayrı** matrise: `s`

Örnek 13: Conway’in Hayat Oyunu

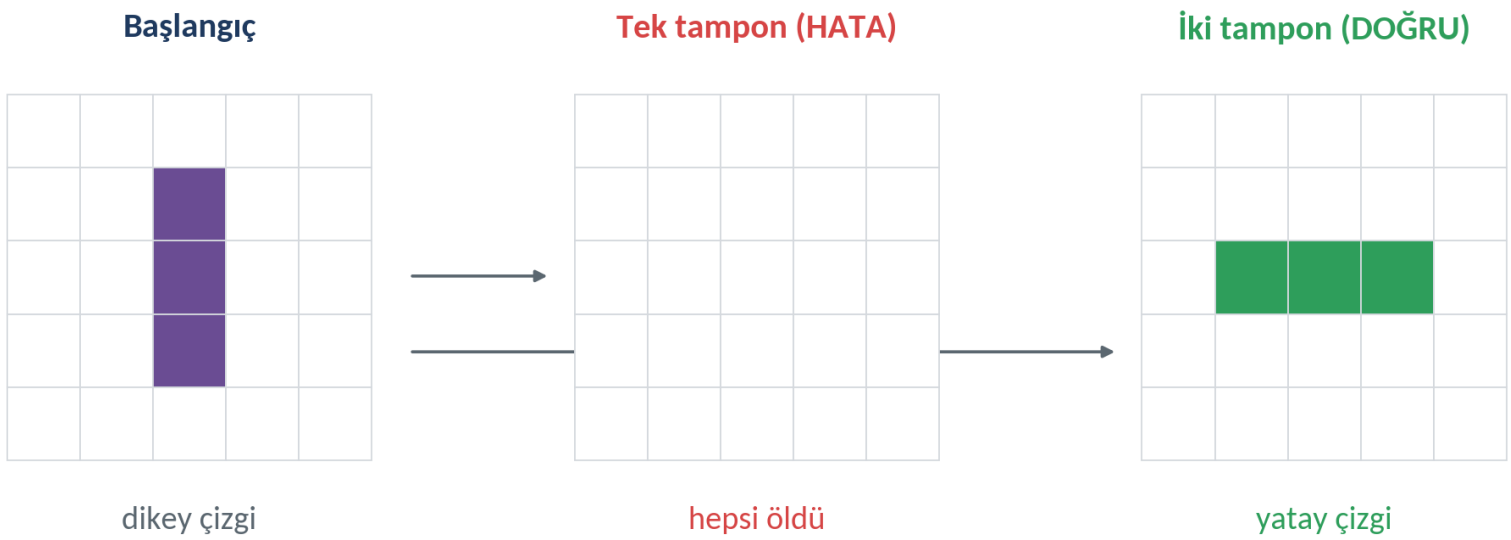
Planör 4 nesilde bir hücre sağ-aşağı kayar; yanıp sönen 2 nesilde bir döner



B3/S23: ölü hücre 3 komşuyla doğar · canlı hücre 2 ya da 3 komşuyla yaşar (H11_13 çıktısı)

- **B3/S23**: 3 komşuyla doğum, 2–3 komşuyla yaşam
- Yeni nesil **eskiden okunur**, ayrı matrise yazılır
- Planör: 4 nesilde **(+1, +1)** kayar
- Yanıp sönen: 2 nesilde bir döner

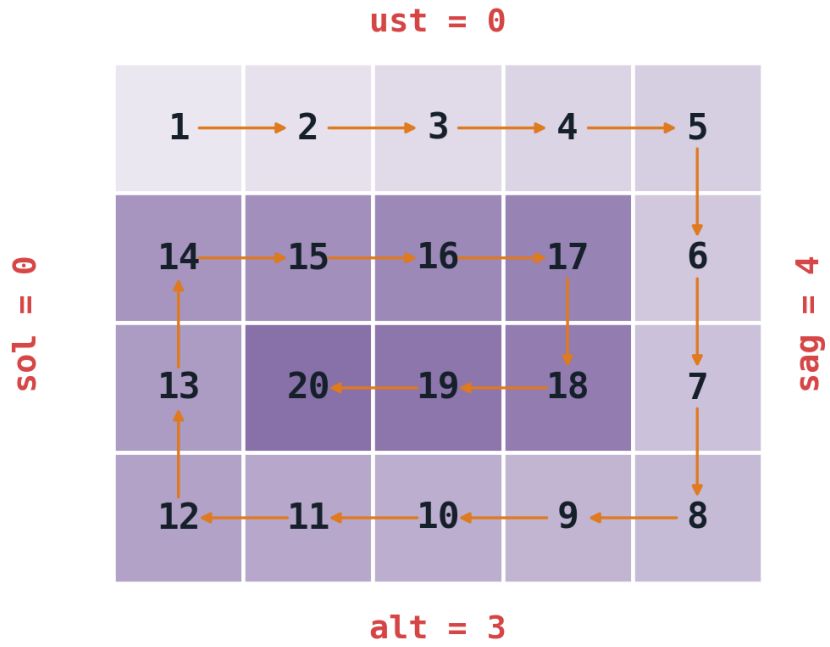
Aynı matrisi okuyup yazarsak: Hata Avcısı 5



tek tamponda güncellenen hücre, komşusunun hesabını bozar

- Tek tamponda güncellenen hücre **komşusunun hesabını bozar**
- Yanıp sönen **tek nesilde ölür** (gerçek çıktı)
- Düzeltme: `eski` → `yeni`, sonra kopyala

Örnek 14: spiral matris



Her tur dört kenar:

→ üst satır, sonra ust++

↓ sağ sütun, sonra sag--

← alt satır (ust <= alt ise)

↑ sol sütun (sol <= sag ise)

`Döngü: ust <= alt && sol <= sag`

Ek denetimler tek satır/sütun kalınca çift saymayı önler

- Dört sınır: `ust, alt, sol, sag`
- Her kenardan sonra o sınır **içeri** çekilir
- `if (ust <= alt)`, `if (sol <= sag)`: tek satır/sütun
- Testler: 3 × 5, 1 × 4, 5 × 1

Hata Avcısı 1: [i][j] yerine [j][i]

HATALI (KASITLI) · hata_avcisi/H11_HA1_ij_karisik.c · satır 18–24

```
18      for (int i = 0; i < OGRENCI; i++) {
19          int toplam = 0;
20          for (int j = 0; j < DERS; j++) {
21              toplam += notlar[j][i];          /* HATA: [i][j] olmalı */
22          }
23          printf("Öğrenci %d ortalaması: %.2f\n", i + 1, (double)toplam / DERS);
24      }
```

İKİ GERÇEK ÇALIŞTIRMA: UYARI YOK, ÇÖP ORTALAMA

```
$ ./ha1      # 1. çalıştırma
Öğrenci 1 ortalaması: 377556008.00
Öğrenci 2 ortalaması: 585199963.67

$ ./ha1      # 2. çalıştırma
Öğrenci 1 ortalaması: -149213400.00
Öğrenci 2 ortalaması: 521213555.00
```

ADDRESSSANİTİZER (GERÇEK, KISALTILMIŞ)

```
AddressSanitizer: stack-buffer-overflow
#0 in main H11_HA1_ij_karisik.c:21
[32, 56) 'notlar' (line 16) <== Memory access at
    offset 56 overflows this variable
```

Düzeltme: `toplam += notlar[i][j];` · bildirim `[OGRENCI][DERS]` ise erişim de `[öğrenci][ders]`

Hata Avcısı 2: kenarda sınır denetimsiz komşu

HATALI (KASITLI) · hata_avcisi/H11_HA2_kenar_komsu.c · satır 14–25

```
14  int komsu_mayin(int m[][SUTUN], int i, int j)
15  {
16      int sayi = 0;
17      for (int di = -1; di <= 1; di++) {
18          for (int dj = -1; dj <= 1; dj++) {
19              if (di != 0 || dj != 0) {
20                  sayi += m[i + di][j + dj];    /* HATA: sınır denetimi yok */
21              }
22          }
23      }
24      return sayi;
25  }
```

GERÇEK ÇIKTI: KENARLARDA ÇÖP

```
$ ./ha2
 -1425438636 54652 32769 5 5
 2 2 0 2 2
 2 2 2 2 -1080098238
 -1080065476 -598411971 -900590494 -1980721563 752526755
```

ADDRESSSANİTİZER (GERÇEK, KISALTILMIŞ)

```
AddressSanitizer: stack-buffer-underflow
#0 in komsu_mayin H11_HA2_kenar_komsu.c:20
[48, 128) 'mayin' (line 29) <== Memory access at
    offset 24 underflows this variable
```

Hata Avcısı 3–6: ileti, neden, çözüm

#	Hata	Gerçek ileti / belirti	Çözüm
3	Parametrede sütun boyutu yok: int m[][]	error: array type has incomplete element type ‘int[]’	int m[][SUTUN]
4	Çarpımda c[i][j] sıfırlanmamış	ikinci çarpım A × I: 20 24 / 46 54 (A değil)	c[i][j] = 0; k döngüsünden önce
5	Hayat Oyunu’nda tek tampon	yanıp sönen tek nesilde ölüyor (.....)	eskiden oku, yeniye yaz
6	İlk değerde iç parantez yok: {1, 2, 4, 5}	warning: missing braces around initializer [-Wmissing-braces]	{{1, 2}, {4, 5}}

MÜHENDİS GÖZÜYLE

Mühendis gözüyle: matris kodu

- **Verimlilik:** veriyi bellekteki sırasıyla dolaşın; 64 MB'da ters sıra -O0'da **2,8×**, -O2'de **15,1×** yavaş
- **Güvenlik:** dizi dışı erişimin en sık yeri **kenarlar**; 1 tabanlı kullanıcı koordinatlarını denetleyin (CERT ARR30-C)
- **Test:** 1 × N, N × 1, köşe, boş ve dolu ızgara; satır/sütun karışıklığı için **dikdörtgen** veri
- **Okunabilirlik:** `#define SATIR/SUTUN`; `i` satır, `j` sütun; bildirime `/* [öğrenci][ders] */`
- **Taşınabilirlik:** VLA MSVC'de yok; büyük matris yerel olursa yığın taşar → `static` (H12: `malloc`)

Mini-OBS v0.11: öğrenci × ders not matrisi

ort[i][d] (öğrenci × ders)

	[0] ALG	[1] MAT	[2] FIZ
ort[0]	79.0	93.0	54.0
ort[1]	46.8	59.8	66.0
ort[2]	88.0	51.2	49.0

vize[i][d], final[i][d] aynı biçimde

DERS_SAYISI = 3 sütun

SATIR → 1B dizi (bitişik)

```
yano_hesapla(ort[i], final[i])
transkript_yaz(…, ort[i])
```

SÜTUN → önce kopyala

```
sutun_kopyala(ort, n, d, s)
dizi_ortalama(s, n) … (v0.10)
```

Yeni: ders istatistikleri · ders × harf frekans matrisi · kredi ağırlıklı YANO ve transkript

- Notlar: `vize[i][d]`, `final[i][d]`, `ort[i][d]`
- Satır `ort[i]` → 1B fonksiyonlar (v0.10)
- Sütun → `sutun_kopyala` → istatistik
- Menü: 1 Ekle · 2 Ders ist. · 3 Transkript · 4 Harf tablosu · 5 Listele · 6 Çıkış
- `make test`: **61 doğrulama**

Mini-OBS v0.11: YANO ve transkript

proje/obs.c · satır 360–371

```c
double yano_hesapla(const double ort_satiri[], const double final_satiri[])
{
    int s = 0, toplam_kredi = 0;
    for (int d = 0; d < DERS_SAYISI; d++) {
        int k = harf_indeksi(yuvarla(ort_satiri[d]), final_satiri[d]);
        int katsayi100 = (int)(harf_katsayisi(k) * 100.0 + 0.5);  /* 400 … 0 */
        s += ders_kredisi(d) * katsayi100;
        toplam_kredi += ders_kredisi(d);
    }
    int yano100 = (2 * s + toplam_kredi) / (2 * toplam_kredi);
    return yano100 / 100.0;
}
```

TRANSKRİPT (T1, GERÇEK, KESİT)

```
Öğrenci no : 2301010045
Ders                  Kredi    Vize   Final    Ort.  Harf  Katsayı
Algoritma ve Prog. I      6   70.00   85.00   79.00    BB     3.00
Matematik I               6   90.00   95.00   93.00    AA     4.00
Fizik I                   5   45.00   60.00   54.00    DC     1.50
YANO          : 2.91
DC notları    : YANO ≥ 2,00 → başarılı sayılır
```

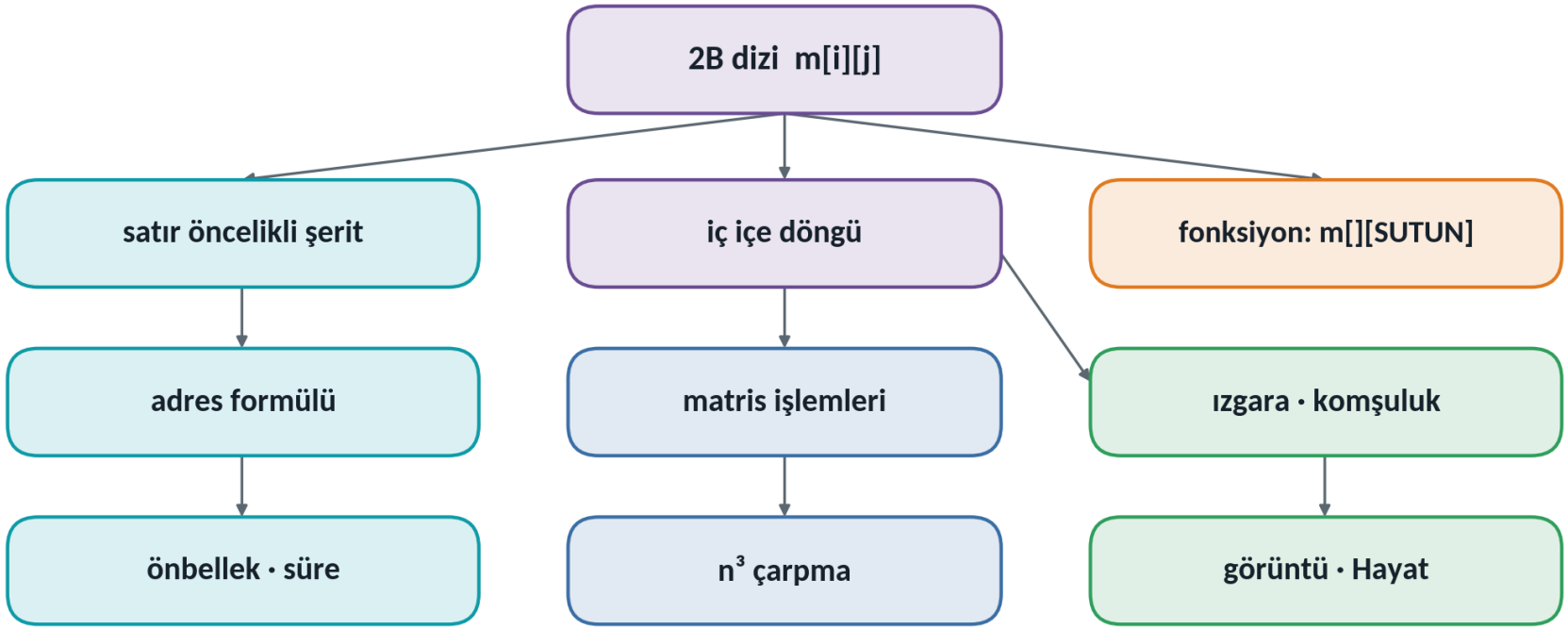
- YANO = Σ kredi × katsayı / Σ kredi, **iki haneye yuvarlanır** (Madde 31/2-a)
- Tamsayıyla: 49,5 / 17 = 2,9118 → **2,91**
- YANO ≥ 2,00 → DC başarılı (30/4-b)

AP-2 KÖPRÜSÜ

AP-2 köprüsü: bu haftanın yapı taşları

- Matris çarpımının n^3 işlem sayısı, AP-2'de polinom karmaşıklık sınıflarının somut örneğidir.
- Izgara komşuluğu, ileride graf dolaşma algoritmalarına (taşkın doldurma, BFS/DFS) açılan kapıdır.
- 2B tablolar, AP-2'de dinamik programlama tablolarının doğal temsilidir.
- Bu derste biçimsel analiz yok: yalnızca **sayıyor** (n^3) ve **ölçüyoruz** (clock)

Özet: haftanın kavram haritası



- **Büyük Soru:** tablo = 2B dizi; bellekte **tek şerit**
- `m[i][j]` = taban + (i × SUTUN + j) × boyut
- Satır satır ardışık, sütun sütun atlayarak: **2,8×** (-O0)

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `int m[4][5];` için `&m[2][3]`, `&m[0][0]`'dan kaç bayt uzaktadır? (sizeof(int) = 4)

2 `void f(int m[][], int n)` neden derlenmez? Nasıl düzeltilir?

3 Hayat Oyunu'nda yeni nesli aynı matrise yazarsak ne olur? Çözüm nedir?

ALGORİTMA ATÖLYESİ · KAPANIŞ

Atölye (75 dk) ve haftaya hazırlık

1. Matris kütüphanesi

30 dk · eşli

`oku`, `yazdir`, `topla`, `devrik`, `carp`: `int m[][AZAMI]` + satır/sütun parametreleri; boyut uyumsuzsa 0 döndür. Test: A · I = A, Örnek 8'in sonucu, dikdörtgen veri, ASan + UBSan.

2. İki oyunculu tic-tac-toe

25 dk

Örnek 9'u genişlet: her hamleden sonra tahtayı çiz, harf/aralık dışı/dolu hücreyi reddet, kazanan ya da berabere, "yeniden oyna?".

3. Görüntü filtreleri

20 dk

Örnek 11'e dikey ayna ve parlaklık (+40, 255'te kırp) ekle; bulanıklaştırmayı "kenarı kopyala" ile yaz, web önizleyicisiyle karşılaştır.

HAFTAYA HAZIRLIK

- **Okuma:** King 8.2; K&R 5.7; Bryant ve O'Hallaron 6
- **Web:** 4 simülasyon + 10 soruluk quiz
- **Ödev:** Alıştırmalar 11.1–11.12 (11.1–11.6'nın kodları `kodlar/alistirmalar/`); Mini-OBS v0.11 (61 doğrulama)
- **H12, İşaretçiler:** `m[1]` bir satırın adresiydi; H8'deki `takas` neden çalışmadı?